

PostgreSQL Row Level Security (RLS)

Table of Contents

1. [Learning Objectives](#)
 2. [What is Row Level Security](#)
 3. [Architecture Diagram](#)
 4. [Enabling RLS](#)
 5. [Policy Syntax](#)
 6. [Policy Examples for Different Access Patterns](#)
 7. [FORCE ROW LEVEL SECURITY](#)
 8. [Policy Combinations and Logic](#)
 9. [Bypassing RLS](#)
 10. [RLS with Application Contexts](#)
 11. [RLS and Views](#)
 12. [Performance Impact](#)
 13. [Common Mistakes](#)
 14. [Best Practices](#)
 15. [Interview Questions](#)
 16. [Exercises and Solutions](#)
-

Learning Objectives

By the end of this module, you will be able to:

- Create RLS policies for at least 5 different access patterns
 - Understand the difference between PERMISSIVE and RESTRICTIVE policies
 - Implement application-level RLS using `SET LOCAL`
 - Bypass RLS safely for superusers and table owners
 - Measure and mitigate RLS performance impact
 - Debug RLS policy failures
-

What is Row Level Security

Row Level Security (RLS) is a fine-grained access control mechanism that restricts which rows individual users can see or modify. It operates at the storage layer — the filter is applied before query execution, not after.

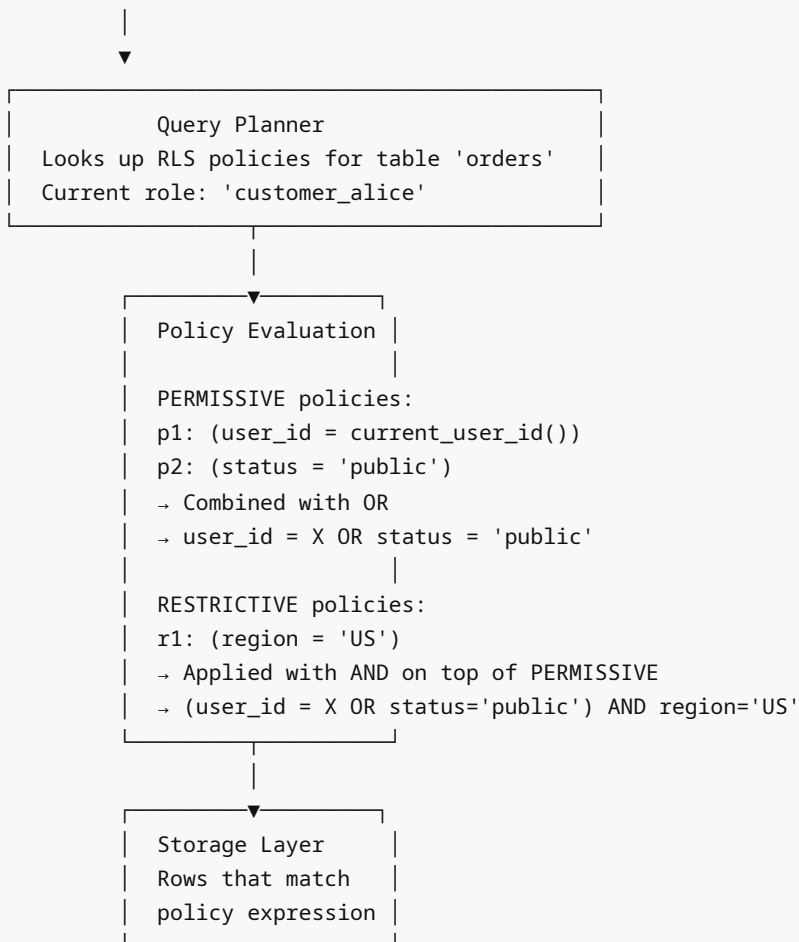
```
Without RLS:
SELECT * FROM orders WHERE ...
→ Returns ALL rows matching WHERE clause

With RLS Policy: "user sees only their own orders":
SELECT * FROM orders WHERE ...
→ PostgreSQL adds: AND user_id = current_user_id()
→ Returns ONLY rows where user_id matches current user
```

RLS is transparent to the application — it doesn't change the SQL but changes which rows are returned.

Architecture Diagram

Query: SELECT * FROM orders



Enabling RLS

```
-- Step 1: Create the table
CREATE TABLE orders (
  id SERIAL PRIMARY KEY,
  user_id INT NOT NULL,
  product_id INT NOT NULL,
  amount NUMERIC(10,2),
  status TEXT DEFAULT 'pending',
  region TEXT DEFAULT 'US',
  created_at TIMESTAMPTZ DEFAULT now()
);

-- Step 2: Enable RLS on the table
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
-- Now: table owner (and superuser) can still see all rows
-- Other users: NO rows visible until policies are created

-- Step 3: Create policies (see below)
```

```

-- Check RLS status
SELECT relname, relrowsecurity, relforcerowsecurity
FROM pg_class
WHERE relname = 'orders';
-- relrowsecurity=true means RLS is enabled
-- relforcerowsecurity=true means even table owner is subject to RLS

-- Disable RLS (removes policy enforcement)
ALTER TABLE orders DISABLE ROW LEVEL SECURITY;

```

Policy Syntax

```

CREATE POLICY policy_name ON table_name
[ AS { PERMISSIVE | RESTRICTIVE } ]
[ FOR { ALL | SELECT | INSERT | UPDATE | DELETE } ]
[ TO { role_name | PUBLIC | CURRENT_USER | SESSION_USER } [, ...] ]
[ USING ( using_expression ) ]      -- Filter for SELECT / UPDATE / DELETE
[ WITH CHECK ( check_expression ) ] -- Filter for INSERT / UPDATE new rows

-- USING: Applied to existing rows (reads and writes affecting existing rows)
-- WITH CHECK: Applied to new row values (writes)
-- If only USING specified for INSERT: also used as WITH CHECK

```

Policy Examples for Different Access Patterns

Pattern 1: Users See Only Their Own Rows

```

-- Multi-tenant: each user sees only their own orders
CREATE TABLE orders (
    id SERIAL PRIMARY KEY,
    user_id INT NOT NULL REFERENCES users(id),
    total NUMERIC(10,2),
    status TEXT
);

ALTER TABLE orders ENABLE ROW LEVEL SECURITY;

-- Policy: SELECT/UPDATE/DELETE only own rows
CREATE POLICY orders_user_isolation ON orders
AS PERMISSIVE
FOR ALL
TO PUBLIC
USING (user_id = current_setting('app.current_user_id')::INT)
WITH CHECK (user_id = current_setting('app.current_user_id')::INT);

-- Application sets the context on each connection:

```

```
-- SET LOCAL app.current_user_id = '42';
-- All subsequent queries filtered to user 42's rows
```

Pattern 2: Role-Based Access (Admins See All)

```
-- Regular users: own rows only
-- Support staff: all rows in their region
-- Admins: all rows

CREATE POLICY row_user_own ON orders
  FOR ALL
  TO customer_role
  USING (user_id = (SELECT id FROM users WHERE username = current_user));

CREATE POLICY row_support_region ON orders
  FOR SELECT
  TO support_role
  USING (region = (SELECT region FROM support_assignments WHERE username =
current_user));

CREATE POLICY row_admin_all ON orders
  FOR ALL
  TO admin_role
  USING (true)      -- Admins see all rows
  WITH CHECK (true); -- Admins can write any row

-- Grant table access
GRANT SELECT, INSERT, UPDATE, DELETE ON orders TO customer_role;
GRANT SELECT ON orders TO support_role;
GRANT ALL ON orders TO admin_role;
```

Pattern 3: Soft Delete (Hide Deleted Rows)

```
-- Add deleted_at column for soft delete
ALTER TABLE orders ADD COLUMN deleted_at TIMESTAMPTZ;

-- Policy: hide deleted rows from regular users
CREATE POLICY hide_deleted ON orders
  AS RESTRICTIVE      -- Must pass this AND any permissive policies
  FOR ALL
  TO PUBLIC
  USING (deleted_at IS NULL); -- Only show non-deleted rows

-- Admin can see deleted rows by bypassing RLS
-- Or by using a separate policy:
CREATE POLICY admin_see_deleted ON orders
  FOR SELECT
  TO admin_role
  USING (true); -- Admins see all including deleted
```

Pattern 4: Status-Based Access

```
-- Published content visible to all; drafts only to owner
CREATE TABLE articles (
  id SERIAL PRIMARY KEY,
  author_id INT,
  title TEXT,
  content TEXT,
  status TEXT DEFAULT 'draft' -- 'draft' or 'published'
);

ALTER TABLE articles ENABLE ROW LEVEL SECURITY;

-- Published: everyone can see
CREATE POLICY view_published ON articles
  FOR SELECT
  TO PUBLIC
  USING (status = 'published');

-- Drafts: only author
CREATE POLICY view_own_drafts ON articles
  FOR SELECT
  TO authenticated_users
  USING (author_id = current_setting('app.user_id')::INT
        OR status = 'published');

-- Write: only author can modify
CREATE POLICY write_own_articles ON articles
  FOR ALL
  TO authenticated_users
  USING (author_id = current_setting('app.user_id')::INT)
  WITH CHECK (author_id = current_setting('app.user_id')::INT);
```

Pattern 5: Time-Based Access (Data Retention)

```
-- Users can only see data from the last 90 days
-- Compliance: data older than 90 days not visible to regular users
CREATE TABLE audit_events (
  id BIGSERIAL PRIMARY KEY,
  user_id INT,
  event_type TEXT,
  event_data JSONB,
  created_at TIMESTAMPTZ DEFAULT now()
);

ALTER TABLE audit_events ENABLE ROW LEVEL SECURITY;

-- Regular users: only recent data
CREATE POLICY recent_data_only ON audit_events
  FOR SELECT
```

```

    TO regular_user_role
    USING (created_at > now() - INTERVAL '90 days'
          AND user_id = current_setting('app.user_id')::INT);

-- Compliance team: all data
CREATE POLICY compliance_all_data ON audit_events
    FOR SELECT
    TO compliance_role
    USING (true);

-- No user can INSERT or modify audit events (append-only)
CREATE POLICY no_modify_audit ON audit_events
    AS RESTRICTIVE
    FOR UPDATE
    TO PUBLIC
    USING (false); -- No one can update audit events

CREATE POLICY no_delete_audit ON audit_events
    AS RESTRICTIVE
    FOR DELETE
    TO PUBLIC
    USING (false); -- No one can delete audit events

```

Pattern 6: Multi-Tenant Schema Isolation

```

-- SaaS multi-tenancy: each tenant sees only their data
CREATE TABLE tenant_data (
    id SERIAL PRIMARY KEY,
    tenant_id UUID NOT NULL,
    data_key TEXT,
    data_value TEXT
);

ALTER TABLE tenant_data ENABLE ROW LEVEL SECURITY;

-- All operations filtered by tenant
CREATE POLICY tenant_isolation ON tenant_data
    AS RESTRICTIVE
    FOR ALL
    TO tenant_user_role
    USING (tenant_id = current_setting('app.tenant_id')::UUID)
    WITH CHECK (tenant_id = current_setting('app.tenant_id')::UUID);

-- Application sets tenant_id for every connection/transaction
-- In PgBouncer transaction mode:
-- SET LOCAL app.tenant_id = '550e8400-e29b-41d4-a716-446655440000';

```

Pattern 7: Hierarchical Access (Department Tree)

```

-- Manager sees employees in their department and sub-departments
CREATE TABLE employee_records (
    id INT PRIMARY KEY,
    name TEXT,
    department_id INT,
    salary NUMERIC,
    manager_id INT
);

ALTER TABLE employee_records ENABLE ROW LEVEL SECURITY;

-- Can see own record
CREATE POLICY see_own_record ON employee_records
    FOR SELECT
    TO employee_role
    USING (id = current_setting('app.employee_id')::INT);

-- Manager can see direct reports
CREATE POLICY see_direct_reports ON employee_records
    FOR SELECT
    TO manager_role
    USING (
        manager_id = current_setting('app.employee_id')::INT
        OR id = current_setting('app.employee_id')::INT
    );

-- HR can see all salary info
CREATE POLICY hr_sees_all ON employee_records
    FOR SELECT
    TO hr_role
    USING (true);

```

FORCE ROW LEVEL SECURITY

By default, table owners and superusers bypass RLS. `FORCE ROW LEVEL SECURITY` makes policies apply even to table owners.

```

-- Make RLS apply even to table owner
ALTER TABLE orders FORCE ROW LEVEL SECURITY;

-- Now even the table creator/owner is subject to RLS policies
-- Only superuser (BYPASSRLS) can bypass

-- This is important for multi-tenant applications where the
-- app database user is typically the table owner

-- Check the setting
SELECT relname, relforcerowsecurity FROM pg_class WHERE relname = 'orders';

```

Policy Combinations and Logic

Multiple PERMISSIVE policies → combined with OR
Multiple RESTRICTIVE policies → combined with AND
Final result → (any PERMISSIVE is true) AND (all RESTRICTIVE are true)

Example:

PERMISSIVE: own_rows (user_id = X)
PERMISSIVE: public_rows (status = 'public')
RESTRICTIVE: not_deleted (deleted_at IS NULL)

Final filter: (user_id = X OR status = 'public') AND deleted_at IS NULL

```
-- Demonstrate policy combination
CREATE TABLE items (
  id INT PRIMARY KEY,
  owner_id INT,
  category TEXT,
  is_public BOOLEAN,
  deleted_at TIMESTAMPTZ
);

ALTER TABLE items ENABLE ROW LEVEL SECURITY;

-- PERMISSIVE: own items
CREATE POLICY own_items ON items
  AS PERMISSIVE FOR SELECT
  USING (owner_id = current_setting('app.user_id')::INT);

-- PERMISSIVE: public items
CREATE POLICY public_items ON items
  AS PERMISSIVE FOR SELECT
  USING (is_public = true);

-- RESTRICTIVE: never show deleted
CREATE POLICY not_deleted ON items
  AS RESTRICTIVE FOR SELECT
  USING (deleted_at IS NULL);

-- Result: (own_items OR public_items) AND not_deleted
-- User sees: their own non-deleted items + public non-deleted items
```

Bypassing RLS

```
-- Method 1: Superuser (ALWAYS bypasses RLS)
-- Connect as postgres superuser → RLS not applied

-- Method 2: BYPASSRLS role attribute
```

```

CREATE ROLE admin_user WITH LOGIN BYPASSRLS PASSWORD 'AdminPass!';
-- Or grant to existing role:
ALTER ROLE admin_user BYPASSRLS;

-- Method 3: Table owner (bypasses unless FORCE ROW LEVEL SECURITY)
-- The user who created the table bypasses RLS by default

-- Method 4: SET ROLE to bypass role
GRANT BYPASSRLS TO admin_function_user;
-- User switches role: SET ROLE admin_function_user;

-- Check bypass status
SELECT rolname, rolbypassrls FROM pg_roles WHERE rolname = 'admin_user';

-- SECURITY: Grant BYPASSRLS sparingly – it's like a master key

```

RLS with Application Contexts

For connection-pooled applications (PgBouncer transaction mode), you cannot use session-level variables. Use `SET LOCAL` within transactions.

```

-- Application: set context per transaction
BEGIN;
SET LOCAL app.current_user_id = '42';
SET LOCAL app.tenant_id = '550e8400-e29b-41d4-a716-446655440000';
-- All queries in this transaction use these values
SELECT * FROM orders; -- filtered to user 42, tenant XXXX
UPDATE orders SET status = 'shipped' WHERE id = 10; -- policy checked
COMMIT;
-- After COMMIT: SET LOCAL values are cleared (safe for pool reuse)

```

```

-- Policy using SET LOCAL context
CREATE POLICY multi_tenant ON orders
FOR ALL
USING (
    tenant_id = current_setting('app.tenant_id', true)::UUID
    -- 'true' means return NULL if not set (vs error)
);

-- Handle missing context gracefully
CREATE POLICY safe_tenant ON orders
FOR ALL
USING (
    CASE
        WHEN current_setting('app.tenant_id', true) IS NULL THEN false
        ELSE tenant_id = current_setting('app.tenant_id')::UUID
    END
);

```

RLS and Views

```
-- SECURITY INVOKER views (default): use the VIEW CALLER's RLS policies
CREATE VIEW my_orders AS
  SELECT id, product_id, amount, status FROM orders;
-- Querying this view applies the caller's RLS policies on 'orders'

-- SECURITY DEFINER views: use the VIEW OWNER's RLS context
CREATE VIEW admin_orders
  WITH (security_invoker = false) -- Owner's policies
AS
  SELECT * FROM orders;

-- SECURITY BARRIER: prevents query optimization from leaking hidden rows
CREATE VIEW safe_orders
  WITH (security_barrier)
AS
  SELECT * FROM orders WHERE status != 'internal';
-- Without security_barrier, a LATERAL join or function call might see 'internal'
rows
-- With security_barrier, the WHERE is applied before any function calls in the
outer query
```

Performance Impact

RLS adds a predicate to every query on the table. This can affect performance if:

1. The policy expression is complex (function calls, subqueries)
2. Indexes don't cover the policy columns
3. Policy prevents index usage

```
-- Create indexes on RLS policy columns
CREATE INDEX idx_orders_user_id ON orders(user_id);
CREATE INDEX idx_orders_tenant_id ON orders(tenant_id);
CREATE INDEX idx_orders_deleted_at ON orders(deleted_at) WHERE deleted_at IS NOT
NULL;

-- Analyze query plan with RLS
-- Connect as a restricted user and EXPLAIN
SET LOCAL app.current_user_id = '42';
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders WHERE status = 'pending';

-- Check if policy is using index:
-- Look for "Index Scan using idx_orders_user_id"
-- vs "Seq Scan" with filter including policy condition

-- Avoid volatile functions in policies (they can't be inlined)
-- BAD:
```

```
CREATE POLICY slow_policy ON orders
  USING (user_id = get_current_user_id()); -- if function is VOLATILE

-- GOOD: use current_setting (stable) or join-based lookup
CREATE POLICY fast_policy ON orders
  USING (user_id = current_setting('app.user_id')::INT); -- uses index
```

Performance Testing

```
-- Measure policy overhead
-- As superuser (bypasses RLS):
EXPLAIN (ANALYZE) SELECT COUNT(*) FROM orders;
-- As restricted user (RLS applied):
SET ROLE restricted_user;
EXPLAIN (ANALYZE) SELECT COUNT(*) FROM orders;
RESET ROLE;

-- The difference in planning and execution time shows RLS overhead
```

Common Mistakes

1. **No policies after enabling RLS** — all rows invisible to regular users (by design, but surprising)
2. **Not indexing policy columns** — full table scans for every query
3. **Using VOLATILE functions in policies** — can't be optimized, applied per-row
4. **Forgetting FORCE ROW LEVEL SECURITY** — table owner bypasses policies
5. **SET without LOCAL in pooled connections** — session setting persists, leaks to next client
6. **PERMISSIVE policy returns no rows** — must return true for at least one policy (combine with OR)
7. **RLS on catalog tables** — not supported
8. **Not testing as the restricted user** — test with `SET ROLE`
9. **Forgetting WITH CHECK for INSERT** — using only USING clause doesn't restrict INSERTs
10. **Row visibility in triggers** — BEFORE triggers see pre-RLS rows; be careful

Best Practices

1. **Index every column used in RLS policy conditions**
2. **Use `current_setting('var', true)`** with the safe-null second argument
3. **Use `SET LOCAL` (not `SET`)** in pooled environments
4. **Test policies explicitly** by running queries as restricted roles
5. **Use `RESTRICTIVE`** for mandatory filters (like soft-delete, tenant isolation)
6. **Document each policy** with a comment explaining its purpose
7. **Apply `FORCE ROW LEVEL SECURITY`** for tables where even the owner should be restricted
8. **Avoid subqueries in policies** — they run for every row; pre-compute in `current_setting`
9. **Use `security_barrier` views** to prevent optimization-related data leaks
10. **Audit policies regularly** with `pg_policies` view

Interview Questions

Q1: What is Row Level Security and how is it different from column-level privileges?

A: Column-level privileges restrict which columns a user can see (vertical filtering). RLS restricts which rows a user can see (horizontal filtering). They complement each other: column privileges say "you can access this column" while RLS says "but only for rows where this condition is true." RLS is transparent to SQL — the same SELECT query returns different rows for different users.

Q2: What is the difference between PERMISSIVE and RESTRICTIVE policies?

A: Multiple PERMISSIVE policies are combined with OR — if any one passes, the row is visible. Multiple RESTRICTIVE policies are combined with AND — all must pass. The final check is: (any PERMISSIVE = true) AND (all RESTRICTIVE = true). Use RESTRICTIVE for mandatory filters that must always apply (soft-delete, tenant isolation), and PERMISSIVE for additive visibility rules.

Q3: How does RLS interact with connection pooling in transaction mode?

A: In transaction mode, session variables (SET var = value) are shared across clients because the underlying PostgreSQL connection is reused. This is dangerous for RLS — one user's context could leak to another. Solution: always use SET LOCAL which is transaction-scoped and automatically reset on COMMIT/ROLLBACK. The application begins each transaction by setting its context variables with SET LOCAL.

Q4: Explain the purpose of FORCE ROW LEVEL SECURITY.

A: By default, the table owner and superusers bypass RLS entirely. FORCE ROW LEVEL SECURITY makes the table owner subject to RLS policies too. Only superusers and roles with BYPASSRLS attribute can bypass. This is important when the application's database role is also the table owner — without FORCE, the app would bypass all policies.

Q5: How would you implement multi-tenancy using RLS?

A: (1) Add a tenant_id column to all tenant-scoped tables. (2) Enable RLS on those tables. (3) Create an RESTRICTIVE policy: USING (tenant_id = current_setting('app.tenant_id')::UUID). (4) The application sets the tenant_id at the start of each transaction: SET LOCAL app.tenant_id = '...'. (5) Use FORCE ROW LEVEL SECURITY so even the table owner is isolated. This provides database-level isolation without separate schemas per tenant.

Q6: Can you use subqueries in RLS policies?

A: Yes, but with caution. Subqueries in RLS policies run for every row evaluation. A subquery like user_id = (SELECT id FROM users WHERE username = current_user) is evaluated for each row in the table, which can be very slow on large tables. Better approaches: pre-compute the value into current_setting (set once per transaction), or use stable/immutable functions that can be planned once.

Q7: How do you debug RLS policy issues?

A: (1) Use SET ROLE restricted_user to test as that user. (2) Run EXPLAIN VERBOSE to see the added policy conditions. (3) Check pg_policies to see all policies on a table. (4) Temporarily disable RLS with ALTER TABLE ... DISABLE ROW LEVEL SECURITY to verify data exists without policies. (5) Check current_setting('app.var', true) to verify context is set correctly.

Q8: What happens when there are no policies on an RLS-enabled table?

A: If RLS is enabled but no policies exist, all rows are DENIED for non-privileged users (the implicit default is deny-all). The table owner and superusers still see all rows (unless FORCE ROW LEVEL SECURITY is set). This is the secure default — you must explicitly create policies to grant access, rather than denying access after the fact.

Exercises and Solutions

Exercise 1: Multi-Tenant E-Commerce

```
-- Implement tenant isolation for a multi-tenant SaaS

-- Tables
CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    tenant_id UUID NOT NULL,
    name TEXT NOT NULL,
    price NUMERIC(10,2),
    active BOOLEAN DEFAULT true
);

CREATE TABLE orders (
    id SERIAL PRIMARY KEY,
    tenant_id UUID NOT NULL,
    customer_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT,
    total NUMERIC(10,2),
    created_at TIMESTAMPTZ DEFAULT now()
);

-- Enable RLS
ALTER TABLE products ENABLE ROW LEVEL SECURITY;
ALTER TABLE products FORCE ROW LEVEL SECURITY;
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE orders FORCE ROW LEVEL SECURITY;

-- Tenant isolation policies
CREATE POLICY tenant_products ON products
    AS RESTRICTIVE FOR ALL
    USING (tenant_id = current_setting('app.tenant_id')::UUID)
    WITH CHECK (tenant_id = current_setting('app.tenant_id')::UUID);

CREATE POLICY tenant_orders ON orders
    AS RESTRICTIVE FOR ALL
    USING (tenant_id = current_setting('app.tenant_id')::UUID)
    WITH CHECK (tenant_id = current_setting('app.tenant_id')::UUID);

-- Usage in application:
BEGIN;
SET LOCAL app.tenant_id = '550e8400-e29b-41d4-a716-446655440000';
SELECT * FROM products; -- Only this tenant's products
INSERT INTO products (tenant_id, name, price)
    VALUES (current_setting('app.tenant_id')::UUID, 'Widget', 9.99);
COMMIT;
```

Exercise 2: Healthcare Data Access (HIPAA-style)

```
-- Medical records: strict access control
CREATE TABLE patient_records (
    id SERIAL PRIMARY KEY,
    patient_id INT NOT NULL,
    record_type TEXT,
    data TEXT,
    created_by INT,
    created_at TIMESTAMPTZ DEFAULT now()
);

ALTER TABLE patient_records ENABLE ROW LEVEL SECURITY;

-- Patient sees their own records
CREATE POLICY patient_own_records ON patient_records
    FOR SELECT TO patient_role
    USING (patient_id = current_setting('app.user_id')::INT);

-- Doctor sees records of their assigned patients
CREATE POLICY doctor_assigned_patients ON patient_records
    FOR SELECT TO doctor_role
    USING (patient_id IN (
        SELECT patient_id FROM doctor_patient_assignments
        WHERE doctor_id = current_setting('app.user_id')::INT
    ));

-- Doctor can create records for their patients
CREATE POLICY doctor_create_records ON patient_records
    FOR INSERT TO doctor_role
    WITH CHECK (
        patient_id IN (
            SELECT patient_id FROM doctor_patient_assignments
            WHERE doctor_id = current_setting('app.user_id')::INT
        )
        AND created_by = current_setting('app.user_id')::INT
    );

-- Admin sees all but cannot modify
CREATE POLICY admin_read_all ON patient_records
    FOR SELECT TO admin_role
    USING (true);
```

Cross-References

- [02 roles and privileges.md](#) — Role-based access control
- [../13 Replication HA/03 logical replication.md](#) — RLS with logical replication
- [06 auditing.md](#) — Auditing RLS-protected tables
- [08 compliance considerations.md](#) — GDPR/HIPAA compliance patterns